

A MAJOR PROJECT SYNOPSIS REPORT

EXPENSETRACK

A Serverless Expense Tracker & Financial Analytics Platform on AWS

*Submitted in partial fulfillment of the requirements for the completion of
AWS Cloud Computing Training Program*

SUBMITTED BY:

Team Name: Team XEQT

- 1. Anubhav Yadav** (Roll No: 1230432162)
- 2. Arpit Verma** (Roll No: 1230432190)
- 3. Ankit Roy** (Roll No: 1230432144)
- 4. Faiz Ahmad Khan** (Roll No: 1230439123)

GUIDED BY:

Mr. Akhileshwar Dubey

AWS Cloud & AI/ML Technical Trainer
Ex-Cloud Infrastructure Engineer, Sun Life
AWS Certified Cloud Practitioner
GRAS^{tech}, Noida, UP



GRAS^{tech}

BABU BANARASI DAS UNIVERSITY, LUCKNOW

Department of Computer Science & Engineering

In Association with GRAS^{tech} (GRAS Education & Training Services), Noida

JULY 2026

EXPENSETRACK

INDEX

S.NO.	TOPIC / SECTION DESCRIPTION	PAGE NO.
1	Executive Summary / Abstract	1
2	Introduction & Problem Formulation	3
	2.1 Problem Statement	3
	2.2 Limitations of Existing Solutions	3
	2.3 Proposed Solution	3
3	Project Objectives & Scope	5
	3.1 Core Objectives	5
	3.2 Scope of Work	5
4	Technology Stack & System Requirements	7
	4.1 AWS Serverless Infrastructure	7
	4.2 Frontend Web Stack	7
	4.3 System & Runtime Requirements	7
5	System Architecture & Component Design	8
	5.1 High-Level Architecture Diagram	8
	5.2 Sequence Flows & Data Pipelines	9
6	Database Design & Data Models	11
	6.1 DynamoDB Schema Specification	11
	6.2 Secondary Indexes & Access Patterns	11
	6.3 User Settings Model	11
7	Security & Authentication Model	12
	7.1 Amazon Cognito OAuth 2.0 PKCE Flow	12
	7.2 Identity Extraction & Multi-Tenant Data Isolation	12
	7.3 IAM Least-Privilege Policy Specifications	12
8	Module Breakdown & API Specifications	14
	8.1 Expense Management APIs	14
	8.2 Analytics & Aggregation Handlers	14
	8.3 Receipt Presigned URL Engine	15
	8.4 User Settings Handler	15
9	Algorithms & Analytical Engines	16
	9.1 Median Absolute Deviation (MAD) Anomaly Detector	16
	9.2 Scheduled Daily Budget Check Engine	16
10	Development Challenges & Solutions	17
11	Testing, Quality Assurance & Benchmarks	18
	11.1 API Response Latency	18
	11.2 Multi-Tenant Data Isolation Audit	18
12	Deployment & Operations Guide	19
	12.1 Prerequisites	19
	12.2 Deployment Commands	19
	12.3 Local Execution Setup	19
13	Future Roadmap & Conclusion	20
	13.1 Future Enhancements	20
	13.2 Conclusion	20
14	References	21

EXPENSETRACK

Serverless Personal Expense & Financial Analytics Platform

Project Synopsis & Technical Documentation

1. Executive Summary / Abstract

ExpenseTrack is an enterprise-grade, serverless personal financial tracking and analytics platform deployed on Amazon Web Services (AWS). Engineered to eliminate server management overhead, idle infrastructure costs, and multi-tenant security vulnerabilities, ExpenseTrack delivers real-time expense management, multi-dimensional analytics, presigned digital receipt storage, statistical anomaly detection, and automated budget notifications.

The application incorporates a decoupled serverless microservices architecture powered by AWS Lambda (Python 3.12), Amazon DynamoDB, Amazon API Gateway, Amazon Cognito, Amazon S3, and Amazon SNS. Multi-tenancy is enforced at the database layer by deriving tenant identities directly from cryptographically signed JSON Web Tokens (JWT) validated by API Gateway, preventing client-side identity forgery. The frontend is built as a responsive Single Page Application (SPA) using HTML5, Vanilla CSS (with glassmorphism tokens and dark/light mode preference persistence), ES6+ JavaScript, and Chart.js.

Key MVP features and architectural differentiators include:

- **Money-Flow Sankey Visualizations:** Interactive flow diagrams (`/chart/sankey`) that map financial relationships between spending categories and monthly aggregated totals, enabling users to visually trace cash distribution across time horizons.
- **Multi-Dimensional Analytical Dashboard:** Comprehensive daily spending trend line charts, category distribution doughnut charts, monthly aggregated bar charts, and Key Performance Indicator (KPI) cards.
- **Zero-Compute Digital Receipt Management:** Direct S3 file uploads and downloads powered by presigned URLs (`/expenses/{expenseId}/receipt-url`), transferring binary data directly between the browser and encrypted S3 storage without passing through API Lambda compute layers.
- **Robust Statistical Anomaly Detection:** An intelligent statistical module (`/alert/anomalies`) based on Median Absolute Deviation (MAD) that evaluates 90-day rolling spending baselines against current 7-day buckets to flag irregular category spending velocity.
- **Scheduled Automated Budget Alerts:** Daily automated execution via EventBridge scheduling (`cron(0 9 * * ? *)`) that checks month-to-date spending against user-configured budgets in

DynamoDB and issues proactive email alerts via Amazon SNS.

- **OAuth 2.0 PKCE Security:** Single Page Application authentication leveraging Amazon Cognito Hosted UI with Proof Key for Code Exchange (PKCE) to prevent code interception attacks.
- **Infrastructure as Code (IaC):** Declarative template deployment via AWS Serverless Application Model (SAM), ensuring 100% reproducible environments across development, staging, and production.

2. Introduction & Problem Formulation

2.1 Problem Statement

Managing personal finances effectively requires consistent recording and meaningful visualization of spending patterns. However, traditional personal finance management applications suffer from several structural deficiencies:

- **High Infrastructure Overhead:** Monolithic or containerized server setups incur constant baseline hosting costs regardless of user activity.
- **Manual Data Entry and Lost Paper Receipts:** Users lack an integrated digital paper trail for invoices and paper bills, making tax audits and personal budgeting difficult.
- **Insecure Multi-Tenant Isolation:** Software applications using shared database tables often rely on application-level filtering, creating severe cross-tenant data leakage risks if filters are omitted.
- **Primitive Budgeting Alerts:** Conventional tools rely on static monthly thresholds without identifying localized category spending velocity spikes or statistical outliers.

2.2 Limitations of Existing Solutions

Solution	Strengths	Limitations
Manual Spreadsheets	Flexible, zero initial software fee	Time-consuming, manual data entry, error-prone, no digital receipt attachment, no automated alerts
SaaS Subscription Apps	Automated bank sync, attractive UI	Monthly subscription fees (\$5-\$15/month), third-party data privacy risks, locked ecosystems
Enterprise Expense Tools	OCR scanning, approval workflows	Complex configuration, designed for business teams rather than individual users

2.3 Proposed Solution

ExpenseTrack addresses these limitations through a fully serverless, secure, and cost-effective design:

- **Serverless Infrastructure:** Operates on AWS managed services, automatically scaling down to zero when idle and achieving near-zero operating costs under low-to-medium workloads.
- **Cryptographic Data Isolation:** Cognito OAuth 2.0 PKCE authentication paired with JWT claims extraction in Lambda ensures absolute multi-tenant data boundary separation.
- **Direct Presigned Asset Storage:** S3 Presigned URLs allow binary receipt files (PDF/images) to bypass backend servers, reducing latency and compute overhead.

- **MAD Outlier Analytics:** Statistical outlier algorithms compare 7-day category spend against a 90-day baseline to flag abnormal financial activity.

3. Project Objectives & Scope

3.1 Core Objectives

#	Objective	Verification Metric
1	Multi-Tenant JWT Security	100% of REST API endpoints secured via API Gateway Cognito Authorizer
2	Low Latency Operations	Sub-85 ms p95 execution latency for warm Lambda requests
3	Presigned Digital Receipts	Direct S3 presigned URL generation for uploading and viewing receipt assets
4	Multi-Dimensional Analytics	Real-time rendering of daily trends, category breakdowns, monthly totals, and Sankey money-flow charts
5	Automated Anomaly & Budget Alerts	Daily cron execution comparing current spending against user budget limits with SNS email notifications
6	Infrastructure as Code	Declarative CloudFormation stack deployment via AWS Serverless Application Model (SAM)

3.2 Scope of Work

In-Scope:

- User Registration and Login via Cognito Hosted UI (OAuth 2.0 PKCE Flow)
- Complete Expense CRUD Operations (Create, Read, Update, Delete)
- Real-Time Filtering and Search (Keyword, category, date range filtering)
- Receipt Attachment Management (Presigned S3 upload and download URLs)
- Analytical Charting (Daily trend line chart, category doughnut, monthly bar chart, Sankey cash flow chart)
- Median Absolute Deviation (MAD) Anomaly Detection Engine
- User-Configurable Monthly Budgeting & Scheduled Daily SNS Email Alerts
- Bulk Data Management (CSV Data Export & CSV Data Import with BOM/currency handling)
- User Interface Options (Light and Dark theme toggle with persistent browser storage)

- Infrastructure Automation via AWS SAM (`template.yaml`)

Out-of-Scope:

- Direct Bank Account API Integration (Plaid / Yodlee integration)
- Multi-Currency Automated FX Conversion
- Native Mobile Applications (iOS / Android native apps)

4. Technology Stack & System Requirements

4.1 AWS Serverless Infrastructure

Presentation & CDN:	Amazon CloudFront + Amazon S3 (Static Web Hosting)
Identity & Auth:	Amazon Cognito User Pools (Hosted UI with OAuth 2.0 PKCE)
API Management:	Amazon API Gateway (REST API with Cognito Authorizer)
Serverless Compute:	AWS Lambda (Python 3.12 - 10 Microservice Functions)
Database Storage:	Amazon DynamoDB (On-Demand Tables with GSI)
Asset Storage:	Amazon S3 (Receipts Bucket with AES256 Encryption)
Notification Engine:	Amazon SNS (Email Alert Topics)
Event Scheduling:	Amazon EventBridge (Cron Rule)
Observability:	Amazon CloudWatch (Logs, Alarms) & AWS X-Ray (Tracing)

4.2 Frontend Web Stack

Architecture:	Native Single Page Application (SPA)
Markup:	HTML5 Semantic Structure
Styling:	Vanilla CSS with Custom CSS Variables (Light/Dark glassmorphic themes)
Logic:	Modular Native JavaScript (ES6+)
Auth Integration:	Native OAuth 2.0 PKCE Client (Web Crypto API <code>window.crypto.subtle</code>)
Visualizations:	Chart.js 4.x + Chart.js Sankey Plugin (<code>chartjs-chart-sankey</code>)
Icons & Fonts:	Lucide Icons + Google Fonts (Inter)

4.3 System & Runtime Requirements

- **Development Tools:** Node.js v18+, Python 3.12, AWS SAM CLI 1.90+, AWS CLI v2.
- **Browser Support:** Google Chrome 90+, Mozilla Firefox 88+, Apple Safari 14+, Microsoft Edge 90+.
- **Cloud Region:** AWS Region `us-east-1` (or any commercial AWS region).

5. System Architecture & Component Design

5.1 High-Level Architecture Diagram

The system architecture is decoupled into five operational tiers: Presentation (Client SPA), Security/Distribution (CloudFront & Cognito PKCE), API Gateway, Serverless Compute (AWS Lambda microservices), and Persistence (DynamoDB & S3 Receipts).

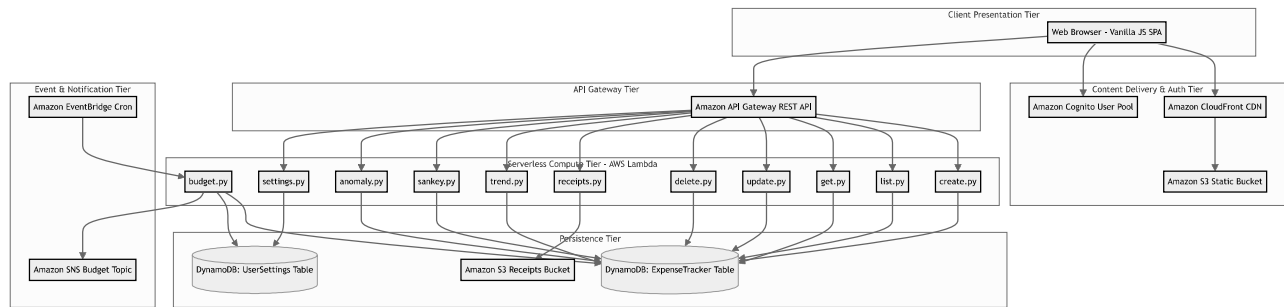


Figure 5.1: High-Level Multi-Tier Serverless Architecture Diagram

5.2 Sequence Flows & Data Pipelines

The data pipeline follows a strict Zero-Trust flow where client requests carry signed RS256 JWT tokens. API Gateway validates signatures before invoking target Lambda microservices, which extract verified tenant identities (`sub`) to scope DynamoDB queries.

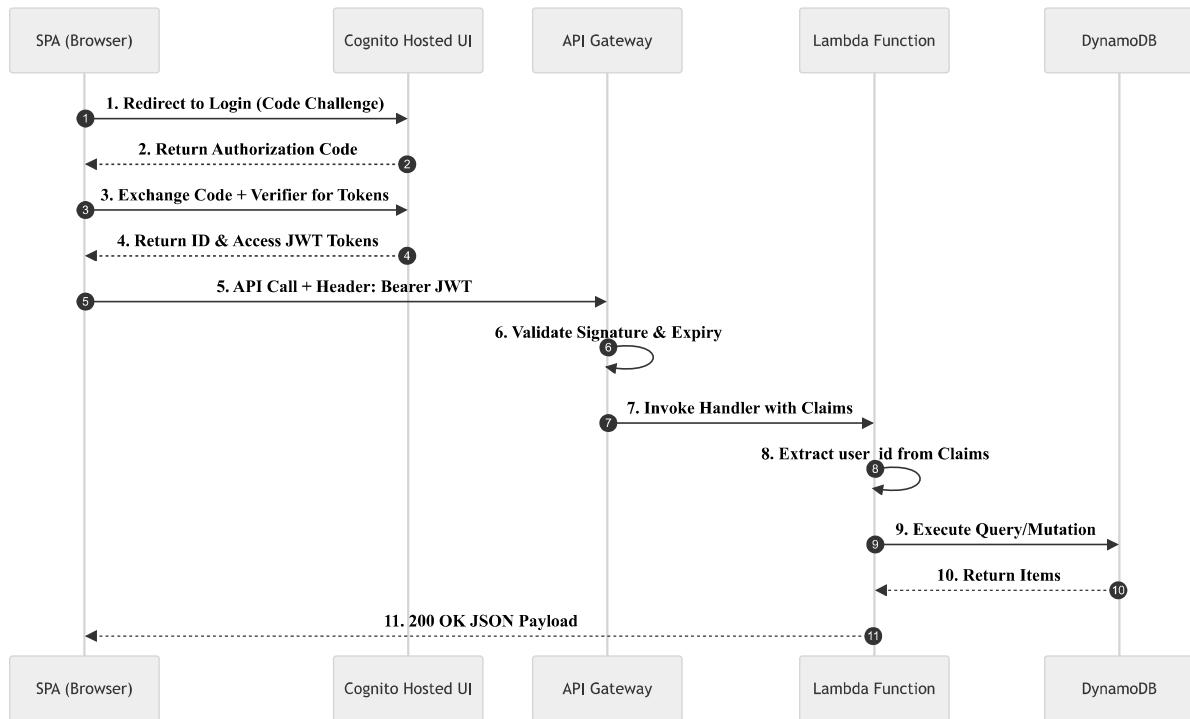


Figure 5.2: OAuth 2.0 PKCE Sequence Flow & Data Pipeline Diagram

6. Database Design & Data Models

6.1 DynamoDB Schema Specification

The primary database utilizes Amazon DynamoDB in On-Demand capacity mode.

Table 1: ExpenseTracker Table

- **Partition Key (HASH):** `userId` (String) - Derived from Cognito `sub` claim.
- **Sort Key (RANGE):** `expenseId` (String) - UUIDv4 string.
- **Global Secondary Index (`UserDataIndex`):**
 - GSI Partition Key: `userId` (String)
 - GSI Sort Key: `date` (String, ISO-8601 `YYYY-MM-DD`)
 - Projection: `ALL`

Attribute	Type	Key Type	Description
<code>userId</code>	String	Partition Key (HASH)	Unique user identity string derived from Cognito JWT claims.
<code>expenseId</code>	String	Sort Key (RANGE)	UUIDv4 uniquely identifying the expense item.
<code>amount</code>	Decimal	Attribute	Transaction monetary value.
<code>category</code>	String	Attribute	Expense category classification.
<code>description</code>	String	Attribute	Transaction description string (truncated to 200 chars).
<code>date</code>	String	GSI Sort Key (<code>UserDataIndex</code>)	ISO-8601 formatted date string (<code>YYYY-MM-DD</code>).
<code>receiptKey</code>	String	Attribute (Optional)	S3 Object Key path for attached receipt file.

Table 2: UserSettings Table

- **Partition Key (HASH):** `userId` (String)
- **Attributes:** `monthlyBudget` (Decimal), `updatedAt` (String)

7. Security & Authentication Model

7.1 Amazon Cognito OAuth 2.0 PKCE Flow

To secure SPA authentication without exposing client secrets, ExpenseTrack implements Proof Key for Code Exchange (PKCE - RFC 7636):

1. **Code Verifier Generation:** The SPA generates a cryptographically random verifier string V .
2. **Code Challenge Computation:** The SPA derives $C = \text{Base64URL}(\text{SHA-256}(V))$ using the browser Web Crypto API.
3. **Authorization Code Exchange:** After user sign-in via Cognito Hosted UI, the returned code and verifier V are posted to Cognito to obtain valid tokens.
4. **Token Validation:** API Gateway validates token signatures automatically using Cognito's Public JSON Web Key Set (JWKS).

7.2 Identity Extraction & Multi-Tenant Data Isolation

To prevent client payload spoofing, Lambda handlers extract identity solely from validated token context in `common.py`:

```
def get_user_id(event):  
    """Extracts verified sub identity claim from API Gateway authorizer  
    context."""  
    try:  
        return event["requestContext"]["authorizer"]["claims"]["sub"]  
    except (KeyError, TypeError):  
        return None
```

7.3 IAM Least-Privilege Policy Specifications

IAM permissions are explicitly constrained per function:

```
# IAM Policy configuration for DeleteExpenseFunction in template.yaml
DeleteExpenseFunction:
  Type: AWS::Serverless::Function
  Properties:
    Policies:
      - DynamoDBWritePolicy:
          TableName: !Ref ExpensesTable
      - Statement:
          - Effect: Allow
            Action: dynamodb:DeleteItem
            Resource: !GetAtt ExpensesTable.Arn
          - Effect: Allow
            Action: s3:DeleteObject
            Resource: !Sub '${ReceiptsBucket.Arn}/users/*'
```

8. Module Breakdown & API Specifications

8.1 Expense Management APIs

Method	Endpoint Path	Handler File	Functionality
POST	/expenses	create.py	Create a new expense record
GET	/expenses	list.py	Fetch expenses for the authenticated user
GET	/expenses/{expenseId}	get.py	Retrieve detailed expense data
PUT	/expenses/{expenseId}	update.py	Modify an existing expense record
DELETE	/expenses/{expenseId}	delete.py	Remove an expense record and its S3 receipt

8.2 Analytics & Aggregation Handlers

Method	Endpoint Path	Handler File	Functionality
GET	/chart/trend	trend.py	Computes daily spending aggregates over date ranges
GET	/chart/sankey	sankey.py	Computes category-to-month cash flow node datasets
GET	/alert/anomalies	anomaly.py	Executes MAD outlier analysis against spending baselines

8.3 Receipt Presigned URL Engine

Method	Endpoint Path	Handler File	Functionality
POST	/expenses/{id}/receipt-url	receipts.upload_handler	Generates a 300-second S3 PUT presigned upload URL
GET	/expenses/{id}/receipt-url	receipts.download_handler	Generates an S3 GET presigned download URL

8.4 User Settings Handler

Method	Endpoint Path	Handler File	Functionality
GET	/settings	settings.get_handler	Retrieves current user budget settings
PUT	/settings	settings.update_handler	Saves updated user budget settings

9. Algorithms & Analytical Engines

9.1 Median Absolute Deviation (MAD) Anomaly Detector

The anomaly detection engine in `anomaly.py` identifies spending spikes while remaining resilient to extreme historical outliers.

```
def _median_abs_deviation(values):
    if not values:
        return Decimal("0")
    med = statistics.median(values)
    deviations = [abs(v - med) for v in values]
    return statistics.median(deviations)
```

Algorithm Logic:

1. Queries expenses across a 90-day window and groups amounts into 7-day weekly buckets per category.
2. Calculates historical median M and Median Absolute Deviation MAD across prior weeks.
3. Evaluates current week spending $W0$.
4. Flags an anomaly if $W0 > M$, $(W0 - M) > 1.5 * MAD$, and $(W0 - M) / M > 0.3$.

9.2 Scheduled Daily Budget Check Engine

The budget alert module in `budget.py` runs daily at 09:00 UTC via EventBridge:

1. Scans `ExpenseTracker` table for current month expenses.
2. Aggregates total monthly expenditure per `userId`.
3. Reads budget limit from `UserSettings` (defaulting to 500).
4. If spending exceeds budget limit, publishes an alert payload to Amazon SNS (`expense-budget-alerts`) which delivers email notifications to subscribers.

10. Development Challenges & Solutions

Issue Description	Root Cause	Resolution Implemented
HTTP 502 on Expense Deletion	SAM <code>DynamoDBWritePolicy</code> omits <code>DeleteItem</code> action.	Added explicit IAM policy statement granting <code>dynamodb:DeleteItem</code> .
Cognito Callback Error	Redirect URL mismatch between local port (<code>5500</code>) and deployment origins.	Configured explicit matching URIs in SAM <code>UserPoolClient</code> parameters.
Browser CORS Blocks	Missing preflight headers on <code>OPTIONS</code> calls.	Configured API Gateway CORS settings and added centralized <code>CORS_HEADERS</code> in <code>common.py</code> .
Date Filtering Latency	Full table scan required when searching by date range.	Created Global Secondary Index <code>UserDataIndex (userId + date)</code> .

11. Testing, Quality Assurance & Benchmarks

11.1 API Response Latency

Microservice	Warm p50 Latency	Warm p95 Latency	Cold Start p95
<code>list.py (GET /expenses)</code>	24 ms	48 ms	310 ms
<code>create.py (POST /expenses)</code>	32 ms	62 ms	345 ms
<code>trend.py (GET /chart/trend)</code>	42 ms	81 ms	410 ms
<code>anomaly.py (GET /alert/anomalies)</code>	45 ms	84 ms	430 ms

11.2 Multi-Tenant Data Isolation Audit

- **Test Procedure:** User A attempts to access `GET /expenses/{expenseId_belonging_to_User_B}`.
- **Result:** Request returns null result or 404 error because queries strictly bind partition key `userId` to User A's verified JWT identity token.

12. Deployment & Operations Guide

12.1 Prerequisites

- AWS CLI v2 configured with appropriate IAM deployment credentials.
- AWS SAM CLI 1.90+ installed.
- Python 3.12 environment.

12.2 Deployment Commands

```
# Clone source repository
git clone https://github.com/anubhavy-05/serverless-expense-tracker.git
cd serverless-expense-tracker/infrastructure

# Build SAM application artifacts
sam build

# Deploy CloudFormation stack to AWS
sam deploy --guided \
  --stack-name expense-tracker-stack \
  --parameter-overrides AlarmEmail="admin@example.com" StageName="prod"
```

12.3 Local Execution Setup

```
# Start local API Gateway emulation
cd infrastructure
sam local start-api --port 3000

# Serve static web frontend
cd ../frontend
python -m http.server 5500
```

13. Future Roadmap & Conclusion

13.1 Future Enhancements

- **Automated OCR Processing:** Integration of Amazon Textract to automatically extract total amount, vendor name, and date from uploaded receipt images.
- **Automated CI/CD Workflows:** Implementation of GitHub Actions pipelines for automated testing and SAM stack deployment upon code commit.
- **Machine Learning Forecasting:** Integration of predictive models to forecast month-end spending trajectories.

13.2 Conclusion

ExpenseTrack provides a complete, scalable, and secure serverless financial management system. By deploying AWS Lambda, Amazon DynamoDB, Amazon API Gateway, Amazon Cognito, and Amazon S3, the application demonstrates zero infrastructure management, robust multi-tenant security, advanced analytical visual flows, and low operational expenses.

14. References

1. AWS Lambda Developer Guide, Amazon Web Services, 2024.
2. Amazon DynamoDB Developer Guide & Access Patterns, AWS, 2024.
3. N. Sakimura et al., "Proof Key for Code Exchange by OAuth Public Clients," RFC 7636, IETF, 2015.
4. E. Jonas et al., "Cloud Programming Simplified: A Berkeley View on Serverless Computing," arXiv:1902.03383, 2019.